EXP 14	Exploring Pro-Code Extension of a Zoho Creator Application Using Zoho Catalyst / Custom Functions

Aim:

To extend the Hotel Room Booking application with pro-code features using custom functions in Zoho Creator and to understand how Zoho Catalyst can be used for advanced logic.

Theory:

Custom functions are reusable blocks of code written in Deluge, NodeJs or Java and called from workflows, buttons, schedules or APIs. Zoho Catalyst is Zoho's serverless pro-code platform providing functions, data store, file storage, cron and APIs for heavy processing.

Procedure:

1. Open the application and go to Workflow > Functions.
2. Click Create Function and check the languages: Deluge, NodeJs, Java.
3. Name the function roomOccupancyReport, choose Deluge, namespace Default, return type void.
4. Write the code to find each room's status, bookings and revenue and the occupancy rate.
5. Save and execute the function and view the log.
6. Create sendCheckinReminders() and call it from the Command Center button.

Program - roomOccupancyReport():

```
void roomOccupancyReport()
{
    totalRooms = 0;
    occupied = 0;
    revenue = 0;
    for each rm in Rooms[ID != 0] sort by Room_Number
    {
        totalRooms = totalRooms + 1;
        bookings = Room_Booking_Form[Room == rm.ID && Status != "Enquiry"];
        roomRevenue = 0;
        for each b in bookings
        {
            roomRevenue = roomRevenue + ifnull(b.Total_amount,0);
        }
        active = Room_Booking_Form[Room == rm.ID && Status == "Checked-In"].count();
        if(active > 0)
        {
            occupied = occupied + 1;
        }
        revenue = revenue + roomRevenue;
        info "Room " + rm.Room_Number + " (" + rm.Room_Type + ") | Status: " +
rm.Room_Status + " | Bookings: " + bookings.count() + " | Revenue: INR " + roomRevenue;
    }
    info "Occupied rooms: " + occupied + " of " + totalRooms + " | Occupancy: " +
(occupied * 100 / totalRooms).round(1) + "% | Total revenue: INR " + revenue;
}
```

Program - sendCheckinReminders():

```
void sendCheckinReminders()
{
    // Email only guests whose Check-In Date is today and whose booking is Confirmed
    sent = 0;
    for each b in Room_Booking_Form[Check_In_Date == zoho.currentdate && Status ==
"Confirmed"]
    {
        sendmail
        [
            from :zoho.adminuserid
            to :b.Email
            subject :"Check-in Reminder - Booking ID " + b.Booking_ID
            message :"Dear " + b.Guest_Name + ",<br><br>This is a reminder that your
check-in is today (" + b.Check_In_Date + ") for a " + b.Room_Type + " room. Check-in
time is 12:00 PM.<br><br>Regards,<br>Front Office"
        ]
        sent = sent + 1;
    }
    info "Check-in reminders sent: " + sent;
}
```

Output:

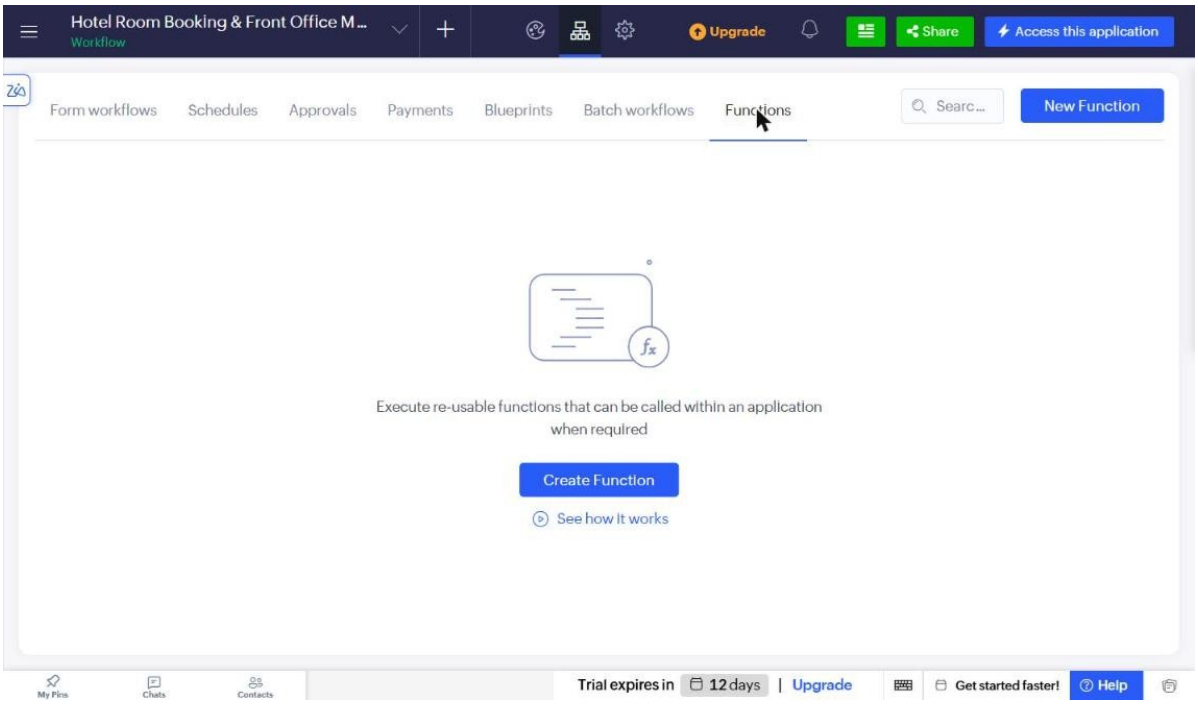


Fig. 14.1: Functions section in the Workflow builder

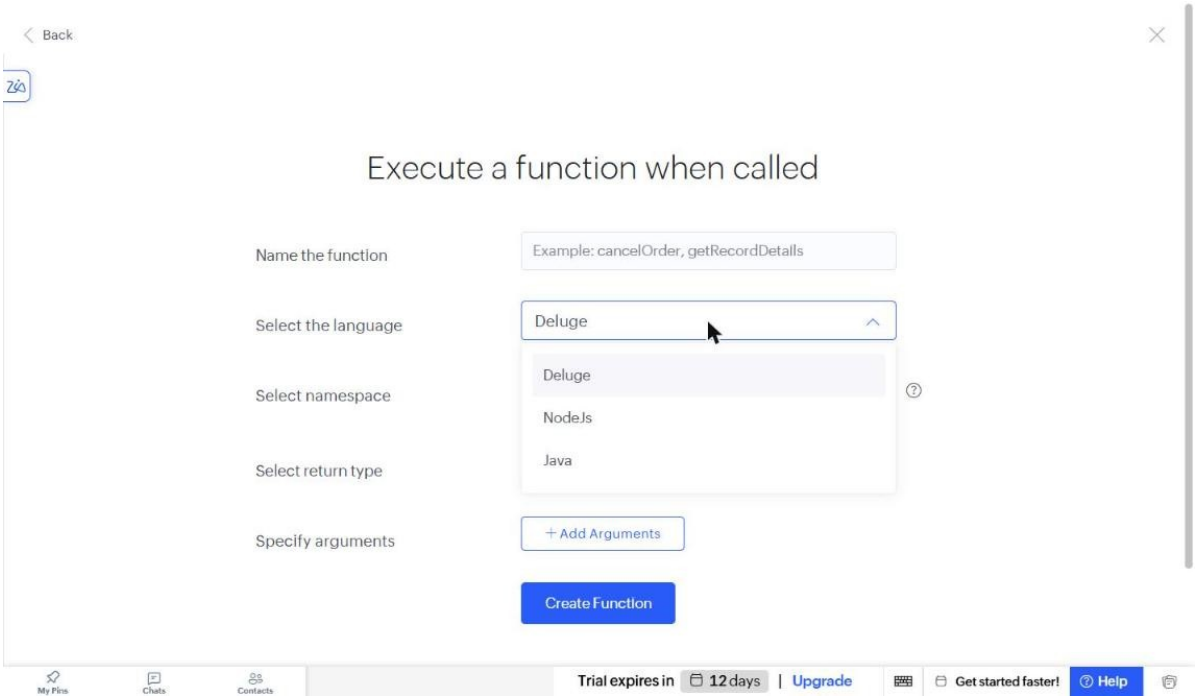


Fig. 14.2: Language options - Deluge, NodeJs and Java

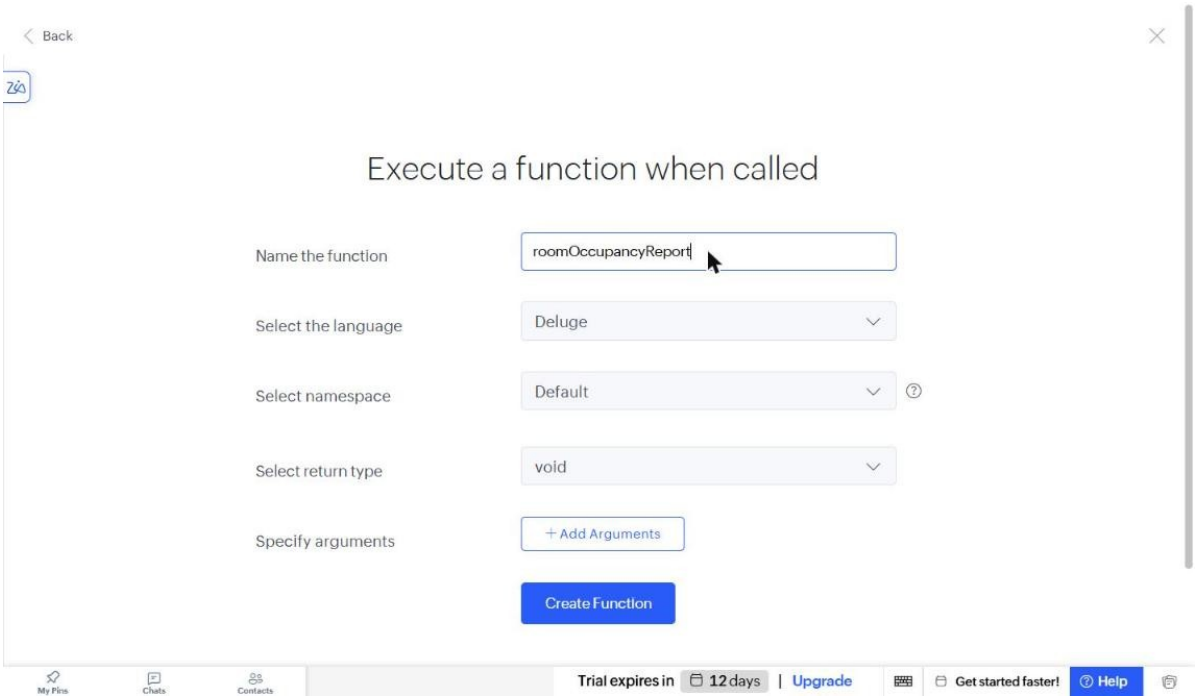


Fig. 14.3: Creating the roomOccupancyReport function

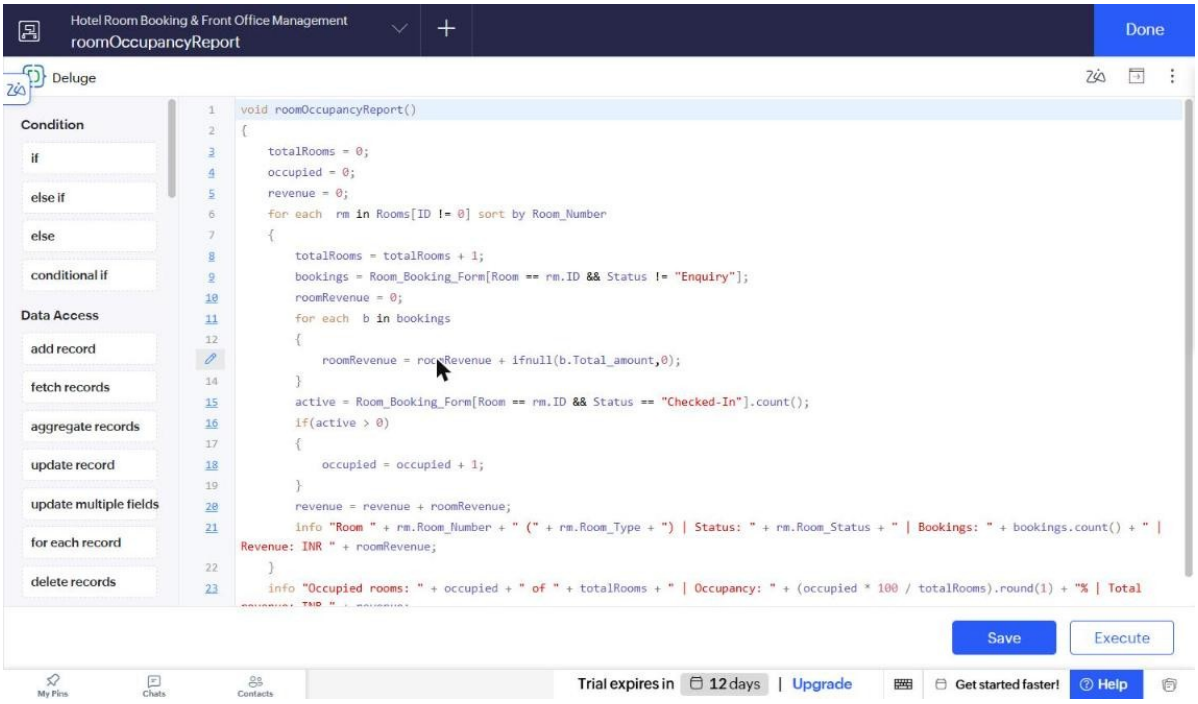


Fig. 14.4: Function code in the editor

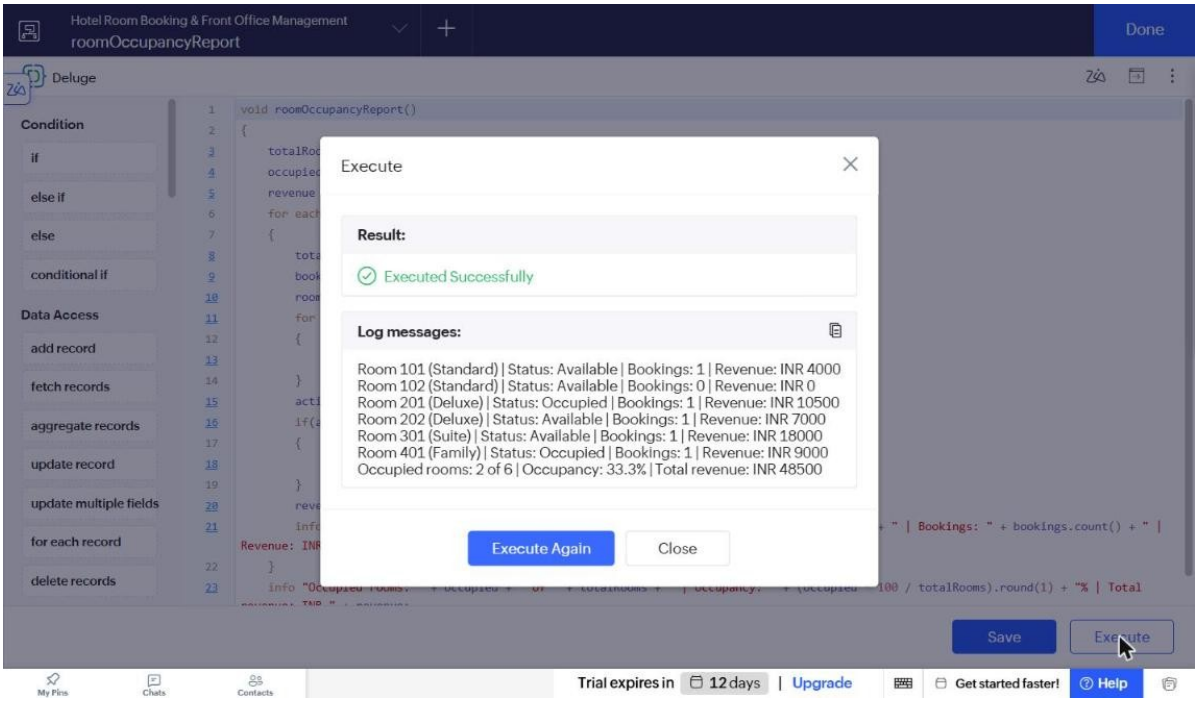


Fig. 14.5: Function executed - log output

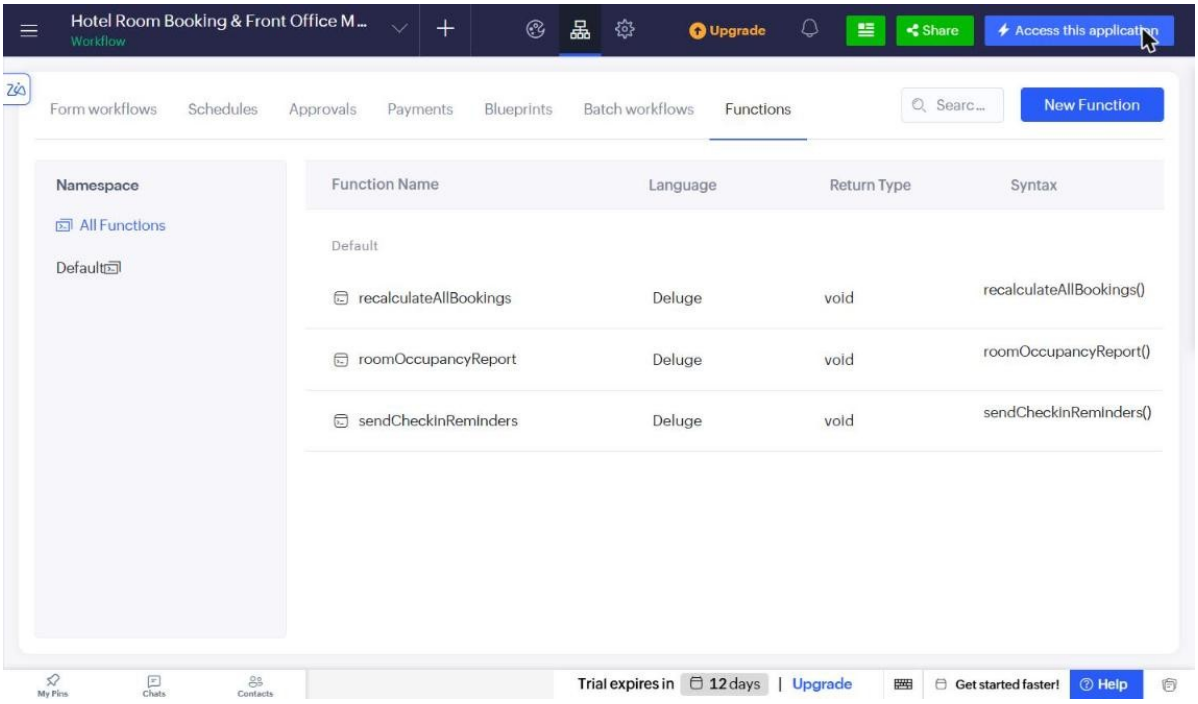


Fig. 14.6: Custom functions of the application

Output of the function:

Room	Type	Status	Revenue (INR)
101	Standard	Available	4000
102	Standard	Available	0
201	Deluxe	Occupied	10500
202	Deluxe	Available	7000
301	Suite	Available	18000
401	Family	Occupied	9000
Total	Occupied 2 of 6 (33.3%)		48500

Result:

Custom Deluge functions were created and executed in Zoho Creator to extend the application with pro-code logic. The same logic can be moved to Zoho Catalyst functions for large data or external integrations.